

项目概述

该项目使用Go语言实现了二次函数多项式回归的可视化工具，通过梯度下降算法拟合二次函数模型 $(y = ax^2 + bx + c)$ ，并实时展示拟合过程。程序能够生成符合目标二次函数 $(y = 1.234x^2 - 3.54x + 2.45)$ 的带噪声样本数据，并通过迭代优化参数 (a) 、 (b) 和 (c) 来逼近真实函数。

💡 核心功能

- 生成带噪声的二次函数样本数据
- 使用梯度下降算法优化模型参数
- 实时可视化展示拟合过程
- 显示训练进度、损失值和参数变化

👁️ 可视化内容

- 样本数据点（白色）
- 真实二次曲线（绿色）
- 当前拟合曲线（橙色）
- 训练统计信息和进度条

多项式回归原理

多项式回归的定义

多项式回归是线性回归的扩展形式，通过引入自变量的高次幂项来拟合非线性关系。对于二次多项式回归，模型形式为：

$$y = ax^2 + bx + c$$

其中：

- (x) 是输入特征
- (y) 是模型预测值
- (a) 、 (b) 和 (c) 是需要通过训练学习的参数

损失函数

使用均方误差（MSE）作为损失函数，衡量预测值与真实值之间的平均差异：

$$MSE = (1/n) \cdot \sum (y_i - (ax_i^2 + bx_i + c))^2$$

其中 (n) 是样本数量, (x_i, y_i) 是第 (i) 个样本的输入和输出。

梯度下降优化

通过计算损失函数对每个参数的梯度, 然后沿梯度反方向更新参数以最小化损失:

$$\begin{aligned}a &= a - lr \cdot \partial(MSE)/\partial a \\b &= b - lr \cdot \partial(MSE)/\partial b \\c &= c - lr \cdot \partial(MSE)/\partial c\end{aligned}$$

各参数的梯度计算如下:

$$\begin{aligned}\partial(MSE)/\partial a &= (2/n) \cdot \sum x_i^2 \cdot (ax_i^2 + bx_i + c - y_i) \\\partial(MSE)/\partial b &= (2/n) \cdot \sum x_i \cdot (ax_i^2 + bx_i + c - y_i) \\\partial(MSE)/\partial c &= (2/n) \cdot \sum (ax_i^2 + bx_i + c - y_i)\end{aligned}$$

其中 (lr) 是学习率, 控制每次参数更新的步长。

代码核心部分解析

1. 数据生成

生成符合目标二次函数并添加高斯噪声的样本数据:

```
func initData(numSamples int) {
    data = make([][]float64, 0, numSamples)
    for i := 0; i < numSamples; i++ {
        // 在xMin到xMax范围内生成随机x值
        x := rand.Float64()*(xMax-xMin) + xMin
        // 生成高斯噪声
        eps := rand.NormFloat64() * Sigma
        // 计算真实y值并添加噪声
        y := ta*x*x + tb*x + tc + eps
        data = append(data, []float64{x, y})
    }
}
```

2. 损失函数计算

实现均方误差 (MSE) 的计算:

```
func Mse(a, b, c float64, points [][]float64) float64 {
    totalError := 0.0
    for _, p := range points {
        x, y := p[0], p[1]
        // 计算每个样本的误差平方并累加
        totalError += math.Pow(y-(a*x+b*x+c), 2)
    }
    // 返回平均误差
    return totalError / float64(len(points))
}
```

3. 梯度下降更新

计算梯度并更新参数 a 、 b 和 c :

```
func StepGradient(a, b, c float64, points [][]float64, lr float64) (float64, float64, float64) {
    aGrad, bGrad, cGrad := 0.0, 0.0, 0.0
    M := float64(len(points))

    for _, p := range points {
        x, y := p[0], p[1]
        err := a*x*x + b*x + c - y // 计算预测误差

        // 计算各参数的梯度
        aGrad += (2 / M) * x * x * err // a的梯度
        bGrad += (2 / M) * x * err     // b的梯度
        cGrad += (2 / M) * err         // c的梯度
    }

    // 沿梯度反方向更新参数
    return a - lr*aGrad, b - lr*bGrad, c - lr*cGrad
}
```

4. 曲线绘制

通过绘制密集线段来近似二次曲线:

```
func drawCurve(screen *ebiten.Image, a, b, c float64, color color.Color) {
    // 使用足够密集的点来近似曲线
    step := (xMax - xMin) / 1000
    for x := xMin; x < xMax; x += step {
        y := a*x*x + b*x + c
        x1Screen := (x - xMin) / (xMax - xMin) * screenWidth
        y1Screen := (yMax - y) / (yMax - yMin) * screenHeight
    }
}
```

```

x2 := x + step
y2 := a*x2*x2 + b*x2 + c
x2Screen := (x2 - xMin) / (xMax - xMin) * screenWidth
y2Screen := (yMax - y2) / (yMax - yMin) * screenHeight

// 绘制相邻两点之间的线段
ebitenutil.DrawLine(screen, x1Screen, y1Screen, x2Screen, y2Screen, color
}
}

```

曲线显示说明

在可视化界面中，可能会出现三条曲线，分别对应不同的阶段和含义：

真实曲线（绿色）

使用预设的真实参数 $(y = 1.234x^2 - 3.54x + 2.45)$ 绘制，代表我们希望模型逼近的目标函数。代码中通过以下语句绘制：

```
drawCurve(screen, ta, tb, tc, trueLineColor)
```

拟合曲线（橙色）

使用当前模型参数 $(y = ax^2 + bx + c)$ 绘制，随着训练进行会逐渐接近真实曲线。代码中通过以下语句绘制：

```
drawCurve(screen, a, b, c, fitLineColor)
```

初始曲线（水平直线）

训练初期，由于参数 (a) 、 (b) 和 (c) 初始值为0，拟合曲线表现为水平线 $(y = 0)$ 。随着迭代进行，该曲线会逐渐转变为二次曲线并逼近真实曲线。

为什么会看到三条曲线？

在训练初期，初始曲线（水平线）与真实曲线差异较大，因此会清晰显示为第三条曲线。随着训练进行，拟合曲线逐渐变化并接近真实曲线，初始曲线的视觉效果会被拟合曲线取代，最终只会看到两条曲线：拟合曲线和真实曲线。

参数说明

以下是影响模型训练和可视化效果的主要参数：

参数	描述	默认值	影响
dataSize	生成的样本数量	2000	样本数量越大，拟合越准确，但计算量也会相应增加
lr	学习率	0.0001	控制参数更新步长，过大会导致算法发散，过小会导致收敛缓慢
numIterations	迭代次数	50000	迭代次数不足可能导致欠拟合，过多会增加训练时间
Sigma	噪声水平	3.0	控制添加到样本数据中的高斯噪声强度，值越大数据越分散，拟合难度越高
updateInterval	可视化更新间隔（秒）	0.001	控制可视化界面的更新频率，值越小动画越流畅，但可能影响性能

使用指南

运行步骤

- 确保安装了Go 1.16或更高版本
- 安装ebiten库：`go get github.com/hajimehoshi/ebiten/v2`

3. 将代码保存为main.go文件
4. 在终端中执行: `go run main.go`
5. 程序将自动打开可视化窗口, 显示初始状态

界面元素说明

坐标轴

水平轴表示自变量x, 垂直轴表示因变量y。坐标范围由xMin、xMax、yMin和yMax参数控制。

数据点

白色圆点表示生成的带噪声样本数据, 它们分布在真实曲线周围。

曲线

绿色曲线代表真实函数, 橙色曲线代表当前拟合结果。

统计区

左上角显示当前参数值(a、b、c)和损失值(MSE), 底部显示训练进度条。

常见问题

拟合曲线不收敛

可能原因: 学习率过大导致算法发散, 或迭代次数不足。解决方法: 尝试减小学习率, 或增加迭代次数。

导出功能异常

确保浏览器允许弹出下载窗口, 或尝试使用Chrome等现代浏览器。

提高拟合精度

可以尝试增加样本量, 调整学习率到更合适的值, 或适当增加迭代次数。